INTERRUPT OPERATIONS

5.1 Introduction

Virtually all computers provide a mechanism by which other modules (I/O, memory) may interrupt the normal processing of the processor. *Interrupt* is signal send by an external device to the processor, to request the processor to perform a particular task or work. Table 5.1 lists the most common classes of interrupts.

Table 5.1: Classes of interrupts

Programs	Generated by some condition that occurs as a result of an instruction execution, such as arithmetic overflow, division by zero, attempt to execute an illegal machine instruction, and reference outside a user's allowed memory space.
Timer	Generated by a timer within the processor to perform certain functions on a regular basis.
I/O	Generated by an I/O controller, to signal normal completion of an operation or to signal a variety of error conditions.
Hardware Failure	Generated by a failure, such as power failure or memory parity error.

notes.arpanadhikari7.com.np

Interrupts are provided primarily as a way to improve processing efficiency. For example, most external devices are much slower than the processor. Suppose that the processor is transferring data to a printer using the instruction cycle we discussed in the previous chapter. After each write operation, the processor must pause and remain idle until the printer catches up. The length of this pause may be on the order of many hundreds or even thousands of instruction cycles that do not involve memory.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Clearly, this is a very wasteful use of the processor. So interrupts are particularly useful when interfacing I/O devices that provide or require data at relatively low data transfer rate.

With interrupts, the processor can be engaged in executing other instructions while an I/O operation is in progress. The interrupt is a process of data transfer whereby an external device or a peripheral can inform the processor that it is ready for communication and it requests attention. The response to an interrupt request is directed or controlled by the microprocessor.

5.2 Polling versus Interrupt

The interrupt process allows the microprocessor to respond to external requests for attention or service on a demand basis and leaves the microprocessor free to perform other tasks. On the other hand, in the polled or status check I/O, the microprocessor remains in a loop doing nothing, until the device is ready for data transfer.

- Each time the device is given a command, for example ``*move the read head to sector 42 of the floppy disk*'' the device driver has a choice as to how it finds out that the command has completed. The device drivers can either poll the device or they can use interrupts.
- Polling the device usually means reading its status register every so often until the device's status changes to indicate that it has completed the request.
- Polling means the CPU keeps checking a flag to indicate if something happens.
- An interrupt driven device driver is one where the hardware device being controlled will cause a hardware interrupt to occur whenever it needs to be serviced.
- With interrupt, CPU is free to do other things, and when something happens, an interrupt is generated to notify the CPU. So it means the CPU does not need to check the flag.
- Polling is like picking up your phone every few seconds to see if you have a call. Interrupts are like waiting for the phone to ring.

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

- Interrupts win if processor has other work to do and event response time is not critical.
- Polling can be better if processor has to respond to an event ASAP; may be used in device controller that contains dedicated secondary processor.

Advantages of interrupt over polling:

- Interrupts are used when we need the fastest response to an event. For example, we need to generate a series of pulses using a timer. The timer generates an interrupt when it overflows and within 1 or 2 sec, the interrupt service routine is called to generate the pulse. If polling were used, the delay would depend on how often the polling is done and could delay response to several msecs. This is thousands times slower.
- Interrupts are used to save power consumption. In many battery powered applications, the microcontroller is put to sleep by stopping all the clocks and reducing power consumption to a few micro amps. Interrupts will awaken the controller from sleep to consume power only when needed. Applications of this are hand held devices such as TV/VCR remote controllers.
- Interrupts can be a far more efficient way to code. Interrupts are used for program debugging.

5.3 Interrupt Structures

A processor is usually provided with one or more interrupt pins on the chip. Therefore a special mechanism is necessary to handle interrupts from several devices that share one of these interrupt lines. There are mainly two ways of servicing interrupts: *polled interrupts* and *daisy chain (vectored) interrupts*.

5.3.1 Polled Interrupts

Polled interrupts are handled by using software, and is slower than hardware interrupts. Here the processor has the general (common) interrupt service routine (ISR) for all devices. The priority of the devices is determined by the

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

order in which the routine polls each device. The processor checks the starting with the highest priority device. Once it determines the source of the interrupt, it branches to the service routine for that device.

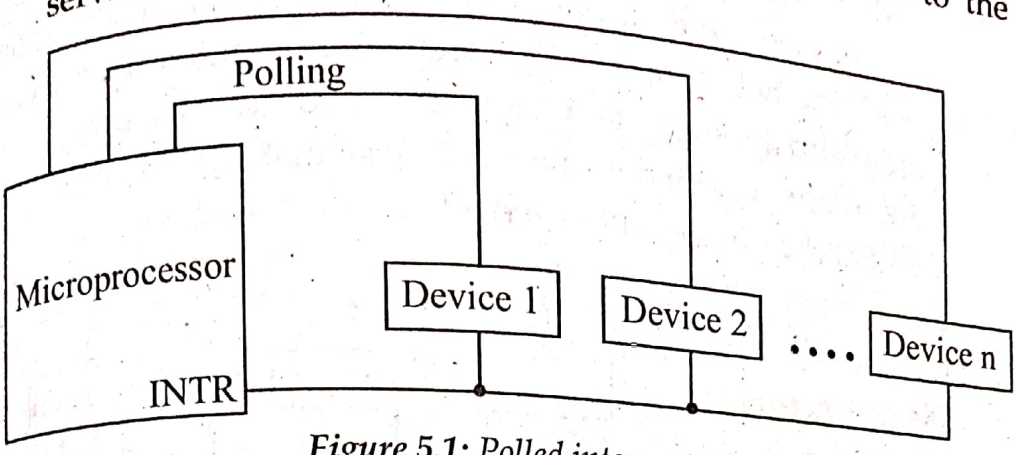


Figure 5.1: Polled interrupt

Here several eternal devices are connected to a single interrupt line (INTR) of the microprocessor. When INTR signal goes up, the processor saves the contents of PC and other registers and then branches to an address defined by the manufactures of the processor. The user can write a program at this address to find the source of the interrupt by starting the polled from highest priority device.

5.3.2 Daisy chain (vectored) interrupt

In polled interrupt, the time required to poll each device may exceed the time to service the device through software. To improve this, the faster mechanism called vectored or daisy chain interrupt is used. Here the devices are connected in chain fashion. This configuration is governed by the priority of the devices. The device with the highest priority is placed first followed by the second highest priority device and so on. When INTR pin goes up, the processor saves its current status and then generates $\overline{\text{INTA}}$ signal to the highest priority device. If this device has generated the interrupt, it will accept the INTA; otherwise it will push $\overline{\text{INTA}}$ to the next priority device until the $\overline{\text{INTA}}$ is accepted by the interrupting device.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

When $\overline{INTA}$ is accepted, the device provides a means to the processor for findings the interrupt address vector using external hardware. The accepted device responds by placing a word on the data lines which becomes the vector address with the help of any hardware through which the processor points to appropriate device service routine. Here no general interrupt service routine need first that means appropriate ISR of the device will be called.

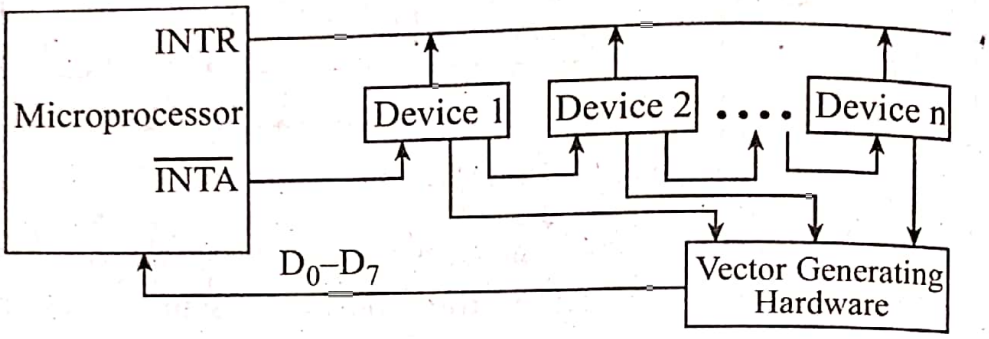


Figure 5.2: Vectored (daisy chain) interrupt

5.4 Interrupt Processing Sequence

The occurrence of an interrupt triggers a number of events, both in the processor hardware and in software. The interrupt driven I/O operation takes the following steps:

- The I/O unit issues an interrupt signal to the processor for exchange of data between them.
- The processor finishes execution of the current instruction before responding to the interrupt.
- The processor tests for an interrupt, determines that there is one, and sends an acknowledgement signal to the device that issued the interrupt.
- The processor now needs to prepare to transfer control to the requested routine called *interrupt service routine* (ISR). To begin, it needs to save information needed to resume the current program at the point of interrupt. The minimum information required is the *program status word* (PSW) and the location of the next instruction be executed, which is contained in the *program counter* (PC). The contents of PSW and PC are pushed onto the stack.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

- The processor loads the program counter with the entry location of the *interrupt-handler program* that will respond to this interrupt, that is, the control is transferred to the interrupt-handler program.
- At this point, the PC and PSW relating to the interrupted program have been saved on the system stack. However, in addition, the contents of the processor registers need to be saved, because these registers may be used by the interrupt-handler. Typically, the interrupt handler will begin by saving the contents of all registers on the stack. Now, the PC is updated to point to the beginning of the *interrupt service routine*.
- The interrupt handler may now proceed to process the interrupt. This will include an examination of status information relating to the I/O operation or other event that caused an interrupt. It may also involve sending additional commands or acknowledgements to the I/O device.
- When interrupt processing is complete, the saved register values are retrieved from the stack and restored to the registers.
- Finally, the PSW and PC values are restored from the stack. As a result, the next instruction to be executed will be from previously interrupted program.

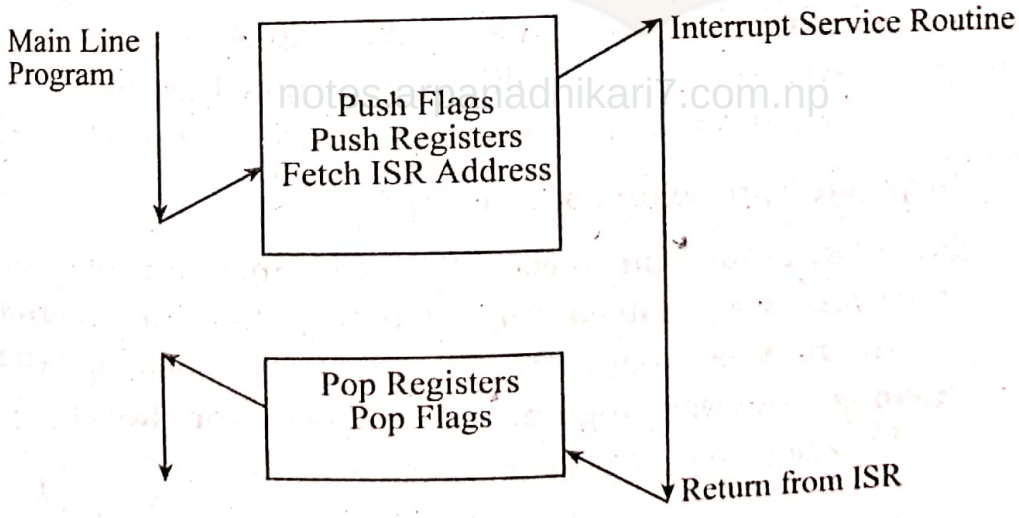


Figure 5.3: Interrupt processing

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

The figure summarizes these steps. The processor pushes the flag register on the stack, disables the INTR input and does essentially an indirect call to the interrupt service procedure. An IRET function at the end of interrupt service procedure returns execution to the main program.

5.5 Multiple Interrupts and Priorities

Upto this point, we have discussed about the occurrence of a single interrupt. Suppose, however, that multiple interrupts can occur. For example, a program may be receiving data from a communications line and printing results. The printer will generate an interrupt every time it completes a print operation. The communication line controller will generate an interrupt every time a unit of data arrives. It is possible for a communications interrupt to occur while a printer interrupt is being processed.

Two approaches can be taken to dealing with *multiple interrupts*. The *first approach* is to disable interrupts while an interrupt is being processed. A *disabled interrupts* simply means that the processor can and will ignore any new interrupt request signal. But this does not take into account of relative priority or time-critical needs. For example, when input arrives from the communications line, it may need to be absorbed rapidly to make room for more input. If the first batch of input has not been processed before the second batch arrives, data may be lost because the buffer on the I/O device may fill and overflow. Therefore, another mechanism has to be implemented – *second approach*. If the processor gets multiple interrupts, then we need to deal these interrupts one at a time and the dealing approaches are:

1. Sequential processing of interrupts

When user program is executing and an interrupt occurs interrupts are disabled immediately. After the interrupt service routine completes, interrupts are enabled before resuming the user program and the processor checks to see if additional interrupts have occurred.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

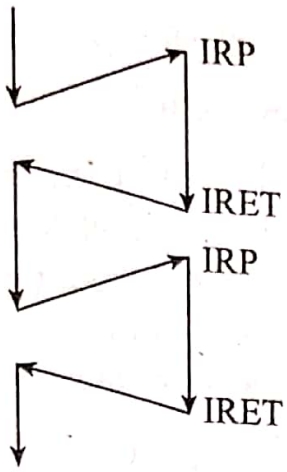


Figure 5.4 Sequential interrupt service

2. **Prioritywise processing of interrupts**

The drawback of sequential processing is that it does not take account of relative priority or time critical needs. The alternative form of this is to define priorities for interrupts and to allow an interrupt of higher priority to cause a lower priority interrupts pause until high priority interrupt completes its function.

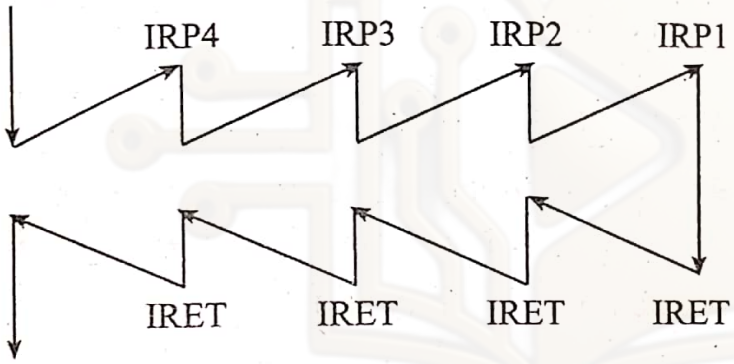


Figure 5.5: Prioritywise interrupt service

Interrupt Service Routine

notes.arpanadhikari7.com.np

An *interrupt service routine* (ISR) is a software routine that hardware invokes in response to an interrupt. ISRs examine an interrupt and determine how to handle it. ISRs handle the interrupt, and then return a logical interrupt value. Its central purpose is to process the interrupt and then return control to the main program. An ISR must perform very fast to avoid slowing down the operation of the device and the operation of all lower priority ISRs. As in procedures, the last instruction in an ISR should be IRET.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

ISR is responsible for doing the following things:

1. **Saving the processor context**

   Because the ISR and main program use the same processor registers, it is the responsibility of the ISR to save the processor's registers before beginning any processing of the interrupt. The processor context consists of the instruction pointer, registers, and any flags. Some processors perform this step automatically.

2. **Acknowledging the interrupt**

   The ISR must clear the existing interrupt, which is done either in the peripheral that generated the interrupt, in the interrupt controller, or both.

3. **Restoring the processor context**

   After interrupt processing, in order to resume the main program, the values that were saved prior to the ISR execution must be restored. Some processors perform this step automatically.

5.6 Interrupts Types

In differentiation with the occurrence of interrupts, they can be classified into various categories based on different parameters:

1. **Hardware and Software Interrupts**

When microprocessors receive interrupt signals through pins (hardware) of microprocessor, they are known as *hardware interrupts*. There are 5 hardware interrupts in 8085 microprocessor namely INTR, RST 7.5, RST 6.5, RST 5.5, TRAP; and 2 hardware interrupts in 8086 microprocessor namely NMI and INTR.

Software interrupts are those which are inserted in between the program which means these are mnemonics of microprocessor. There are 8 software interrupts in 8085 microprocessor namely RST 0, RST 1, RST 2, RST 3, RST 4, RST 5, RST 6, RST 7 and 256 software interrupts in 8086 microprocessor namely INT 00H to INT FFH.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

2. Maskable and Non-Maskable Interrupts

Maskable interrupts are those which can be enabled or disabled by the microprocessor by executing instructions such as EI and DI. These interrupts are either edge-triggered or level-triggered, so they can be disabled. INTR, RST 7.5, RST 6.5, RST 5.5 are maskable interrupts in 8085 microprocessor.

Non-maskable interrupts are those which cannot be enabled or disabled by microprocessor by executing instructions. TRAP is a non-maskable interrupt. It consists of both level as well as edge triggering and is used in critical power failure conditions.

3. Vectored and Non-Vectored Interrupts

Vectored interrupts are those which have fixed vector address (starting address of sub-routine) and after executing these, program control is transferred to that address.

Non-vectored interrupts are those in which vector address is not predefined. The interrupting device gives the address of sub-routine for these interrupts. INTR is the non-vectored interrupt.

4. External and Internal Interrupts

An *external interrupt* is a computer system interrupt that happens as a result of outside interference, whether that's from the user, from peripherals, from other hardware devices or through a network.

notes.arpanadhikari7.com.np

An *internal interrupt* is a specific type of interrupt that is caused by instructions embedded in the execution instructions of a program or process. Typically, internal interrupts resist changes by users, and happen "naturally" or "automatically" as a processor works through program instructions, rather than being caused by external events or network connections. These are indicated internally by exceptional conditions such as overflow, divide by zero, and execution of illegal op-code. The user usually writes a service routine to take correction measures and to provide

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

an indication in order to inform the user that exceptional condition has occurred.

5.7 Interrupts in 8085

The 8085 interrupt process can be described as follows:

1. The interrupt process should be enabled using instruction EI which sets the interrupt enable flip-flop. The instruction DI resets the flip-flop and disables the interrupt process.
2. When the microprocessor is executing a program, it checks the INTR line during the execution of each instruction at the 2nd T-state of last machine cycle.
3. If the line INTR is high and the interrupt is enabled, the microprocessor completes the current instruction, disables the interrupt enable flip-flop, and sends an interrupt acknowledge signal ($\overline{\text{INTA}}$).
4. The program control is transferred to the interrupt service routine (ISR). Now the processor saves the memory address of the next instruction and the contents of registers on the stack.
5. The processor services the interrupt using interrupt service routine. At the end of the subroutine, the RET instruction retrieves the memory address where the program was interrupted and continues the execution.

Types of 8085 Interrupts

The 8085 microprocessor has multilevel interrupt system. It supports two types of interrupts: *hardware interrupts* and *software interrupts*.

1. Software Interrupts

8085 allows some special instructions to interrupt the main program. After executing these instructions, 8085 completes the execution of instruction it is currently executing and transfers the program control to an interrupt service routine. Upon completion of the service routine, execution returns to

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

the main program. These types of interrupts where instructions cause interrupt requests are known as software interrupts.

Hardware Interrupts

Some PINs on the 8085 allow peripheral devices to interrupt the main program for I/O operations. When an interrupt occurs, 8085 completes the execution of instruction it is currently executing and transfers the program control to an interrupt service routine that services the peripheral devices. Upon completion of the service routine, execution returns to the main program. These types of interrupts where MPU PINs are used to receive interrupt requests are known as hardware interrupts.

Software Interrupts

The 8085 has eight software interrupts from RST 0 to RST 7. The vector address for these interrupts can be calculated as follows.

Interrupt number × 8 = vector address

For example, 5 × 8 = 40 = 28H

Vector address for interrupt RST 5.5 is 0028H

Table 5.9 shows the vector addresses of all interrupts.

Table 5.9: *Vector addresses for software interrupts*

Instruction	Hexcode	Vector Address
RST 0	C7H	0000H
RST 1	CFH	0008H
RST 2	D7H	0010H
RST 3	DFH	0018H
RST 4	E7H	0020H
RST 5	EFH	0028H
RST 6	F7H	0030H
RST 7	FFH	0038H

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Hardware Interrupts

The 8085 has five hardware interrupts.

Table 5.10 8085 hardware interrupts

S.N.	Interrupt	Trigger	Priority	Maskable	Vector Address
1	TRAP	Edge and Level	1st	No	0024H
2	RST 7.5	Edge	2nd	Yes	003CH
3	RST 6.5	Level	3rd	Yes	0034H
4	RST 5.5	Level	4th	Yes	002CH
5	INTR	Level	5th	Yes	-

When any of these PINs except INTR is active, the internal control circuit of 8085 produces a CALL to a predetermined memory location. This memory location, where the service routine starts is referred to as vectored location and such interrupts are known as vectored interrupts. The INTR is not a vectored interrupt, it receives the address of ISR from the external device.

In 8085, all interrupts except TRAP are maskable. When logic signal is applied to a maskable interrupt input, the 8085 is interrupted only if that particular input is enabled. These interrupts can be enabled or disabled under program control. If disabled, 8085 disables an interrupt request. The interrupt TRAP is non-maskable that means it is under program control.

notes.arpanadhikari7.com.np

TRAP

This interrupt is a non-maskable vectored interrupt, it is unaffected by any mask or interrupt enable. TRAP has the highest priority and is edge and level triggered. This means that the TRAP must go high and remain high until it is acknowledged. It cannot be acknowledged again until it makes a transition from high to low to high. This avoids false triggering caused by noise and transients. When this interrupt is triggered, the program control is transferred to location 0024H without an external hardware or interrupt enable

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

instruction EI. TRAP is generally used for critical events such as sudden power failure and emergency shut-off

There are two ways to clear TRAP interrupt.

1. By resetting microprocessor that is by giving low signal on $\overline{\text{RESETIN}}$ (external signal).

2. By giving a high TRAP acknowledge (internal signal).

RST 7.5

The RST 7.5 interrupt is a maskable vectored interrupt. It has the second highest priority. It is positive edge sensitive and can be triggered with a short pulse that is input goes to high and no need to maintain high state until it recognized. It can be enabled with two instructions EI (Enable Interrupt) and SIM (Set Interrupt Mask). When this interrupt is triggered, the program control is transferred to location 003CH.

RST 7.5 can be disabled by

1. Using DI instruction.
2. System or processor reset.
3. After reorganization of interrupt.

RST 6.5 and RST 5.5

The RST 6.5 and RST 5.5 both are maskable vectored interrupts. The RST 6.5 has the third priority whereas the RST 5.5 has the fourth priority. These are level triggered that is input goes to high and stay high until it is recognized. It can be enabled with two instructions EI (Enable Interrupt) and SIM (Set Interrupt Mask). When RST 6.5 interrupt is triggered, the program control is transferred to location 0034H and when RST 6.5 interrupt is triggered, the program control is transferred to location 002CH.

They can be disabled by

1. By using DI or SIM instructions.
2. System or processor reset.
3. After reorganization of interrupt.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

INTR

The INTR is a maskable non-vectored interrupt. It has the lowest priority. It can be enabled by EI (Enable Interrupt) instruction. It is a level triggered interrupt that is input goes to high and it is necessary to maintain high state until it recognized. After receiving $\overline{\text{INTA}}$ signal, it has to supply the address of ISR.

It can be disabled by

1. By using DI or SIM instructions.
2. System or processor reset.
3. After reorganization of interrupt.

The following sequence of events occurs when INTR signal goes high.

1. The 8085 checks the status of INTR signal during execution of each instruction.
2. If INTR signal is high, then 8085 complete its current instruction and sends active low interrupt acknowledge signal $\overline{\text{INTA}}$, if the interrupt is enabled.
3. In response to $\overline{\text{INTA}}$ signal, external logic places an instruction opcode on the data bus. In the case of multi-byte instruction, additional interrupt acknowledge machine cycles are generated by the 8085 to transfer the additional bytes into the microprocessor.
4. On receiving the instruction, the 8085 save the address of next instruction on stack and execute received instruction.

8085 Interrupt Instructions

Maskable interrupts are enabled or disabled that is masked or unmasked using program control. There are four instructions used for program control in 8085 microprocessor namely EI, DI, SIM and RIM.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

1. **EI (Enable Interrupt)**

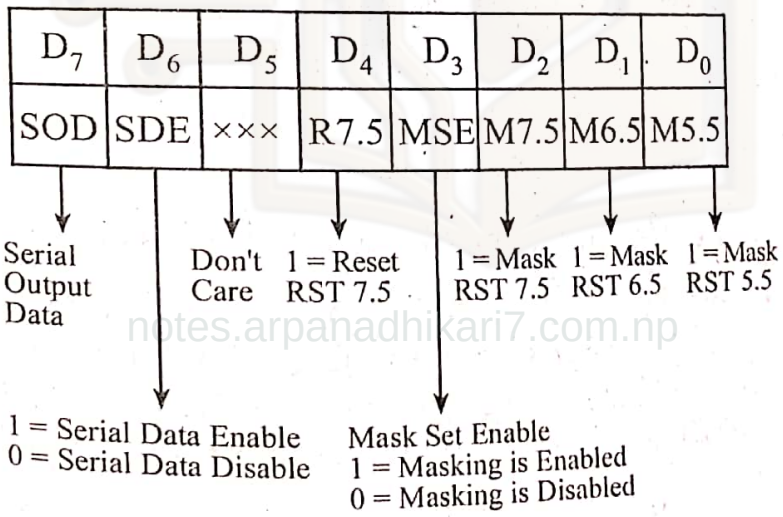
The EI instruction sets the interrupt enable flip-flop. Thus RST 7.5, RST 6.5, RST 5.5 and INTR are enabled using EI instruction. When any interrupt is acknowledged or system reset, interrupt enable flip-flop resets and disables all interrupts. To enable interrupt in further process it is necessary to execute EI instruction within interrupt service routine. It is 1-byte instruction and no flags are affected.

2. **DI (Disable Interrupt)**

DI instruction resets interrupt enable flip-flop. Thus it disables RST 7.5, RST 6.5, RST 5.5 and INTR interrupts. It is of 1-byte instruction and no flags are affected.

3. **SIM (Set Interrupt Mask)**

This instruction is used to set interrupt mask and to send serial output. It transfers the content of accumulator to interrupt control logic and serial I/O ports. Thus it is necessary to load appropriate contents in the accumulator before execution of SIM instruction. It is of 1-byte instruction and no flags are affected. The format of SIM instruction is shown in figure below.



notes.arpanadhikari7.com.np

***Figure 5.11** SIM instruction format*

Actually, SIM does the following three tasks:

1. Masks the interrupts, bits 0 to bit 2 set/reset the mask bits in Interrupt mask register. Bit 0 masks RST 5.5, bit 1 masks RST 6.5 and bit 3 masks RST 7.5. Bit 3 enables/disables the masking control of bits 0 – 2.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

2. Resets RST 7.5, bit 4 is used to reset it whether RST 7.5 is masked or not. This is mainly used to overwrite RST 7.5 without serving it.

3. Implements serial I/O. If bit 6 = 1 is used to enable serial I/O and bit 7 is used to transmit serial output data bit.

3. RIM (Read Interrupt Mask)

The RIM instruction loads the status of the interrupt mask, the pending interrupts and the contents of serial input data line, SID, into accumulator. Thus it is possible to monitor the status of interrupt mask, pending interrupts and serial input. The format of RIM instruction is shown in figure below.

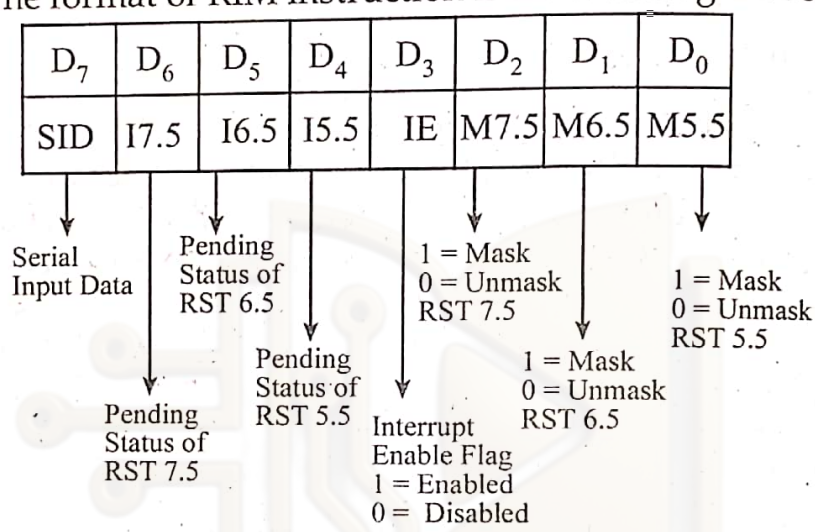


Figure 5.12 RIM instruction format

Actually, RIM does the following four tasks:

1. Checks whether RST 5.5, RST 6.5, and RST 7.5 are masked or not; bit 0 to bit 2 show the current setting of the mask status of interrupts RST 5.5, RST 6.5 and RST 7.5. logic 1 indicates the interrupt is masked.

2. Checks whether interrupts are enabled or not; bit 3 shows the status of interrupt enable flip flop. Logic 1 indicates that the interrupts are enabled.

3. Checks whether RST 5.5, RST 6.5, and RST 7.5 interrupts are pending or not; bit 4 to bit 6 show the status of pending interrupts RST 5.5, RST 6.5, and RST 7.5. Logic 1 indicates the interrupt is pending.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

4. Performs serial input of data; bit 7 is used for Serial Data Input.

5.8 Interrupts in 8086

An 8086 interrupt can come from any one of three sources. One is an external signal applied to NMI pin or to INTR pin, and are referred to as *hardware interrupts*. Another source of an interrupt is execution of the interrupt instruction and is referred to as *software interrupt*. The third source of interrupt is some error conditions produced during the execution of an instruction.

The 8086 interrupt process can be described as follows:

1. At the end of each instruction cycle, the 8086 checks to see if any interrupts have been requested.
2. If there is any interrupt, the 8086 pushes the flag register on the stack.
3. The 8086 disables INTR input by clearing the interrupt flag (IF) and resets the trap flag (TF).
4. The contents of current code segment register and instruction pointer are pushed on the stack.
5. Then, the program control is transferred to interrupt service routine. An IRET instruction at the end of ISR returns execution to the main program by retrieving the contents which were pushed on the stack before.

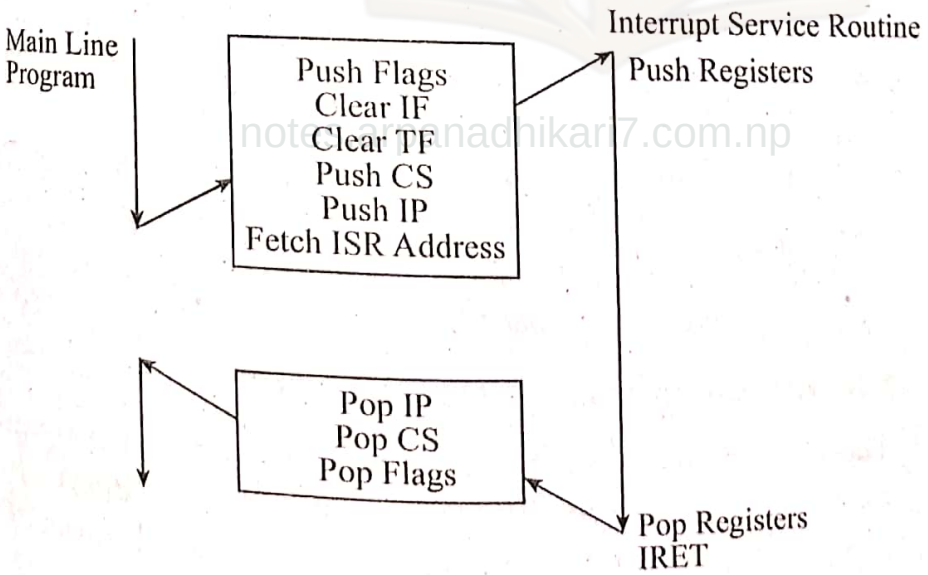


Figure 5.12 8086 interrupt response

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

8086 Interrupt Types

There are two types of Interrupts in 8086. They are:

1. **Hardware Interrupts** (External Interrupts)

Hardware interrupts are those interrupts which are caused by any peripheral device by sending a signal through a specified pin to the microprocessor. There are two hardware interrupts in 8086 microprocessor:

i. **NMI (Non-Maskable Interrupt)** – It is a single pin non-maskable hardware interrupt which cannot be disabled. It is the highest priority interrupt in 8086 microprocessor. After its execution, this interrupt generates a TYPE 2 interrupt. IP is loaded from word location 00008H and CS is loaded from the word location 0000AH.

ii. **INTR (Interrupt Request)** – It provides a single interrupt request and is activated by I/O port. This interrupt can be masked or delayed. It is a level triggered interrupt. It can receive any interrupt type, so the value of IP and CS will change on the interrupt type received. The INTA is the response pin for INTR request.

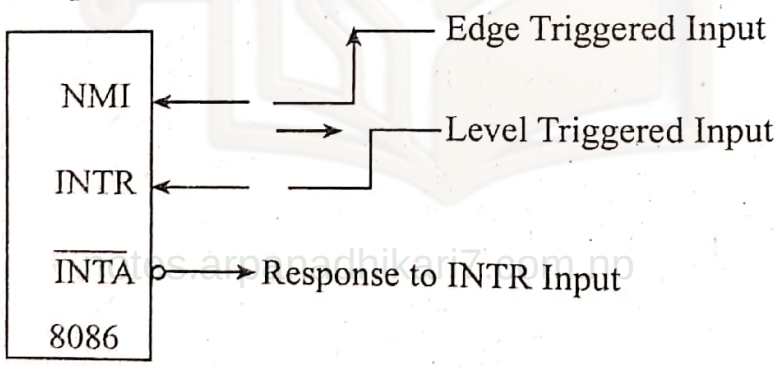


Figure 5.13 8086 hardware interrupts

2. **Software Interrupts** (Internal Interrupts)

These are instructions that are inserted within the program to generate interrupts. There are 256 software interrupts in 8086 microprocessor. The instructions are of the format INT type where type ranges from 00H to FFH. The starting

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

address ranges from 00000H to 003FFH. These are 2 byte instructions. IP is loaded from (type × 04H) and CS is loaded from the next address given by (type × 04H + 02H). Some important software interrupts are:

- **Type 0 : Divide by Zero Interrupt**

  When the quotient from either a DIV or IDIV instruction is too large to fit in the result register; 8086 Interrupt Types will automatically execute type 0 interrupt.

- **Type 1 : Single Step Interrupt**

  The type 1 interrupt is the single step trap. In the single step mode, system will execute one instruction and wait for further direction from user. Then user can examine the contents of registers and memory locations and if they are correct, user can tell the system to execute the next instruction. This feature is useful for debugging assembly language programs.

- **Type 2 : Non-Maskable Interrupt (NMI)**

  As the name suggests, this interrupt cannot be disabled by any software instruction. This interrupt is activated by low to high transition on 8086 NMI input pin. In response, 8086 will do a type 2 interrupt.

- **Type 3 : Breakpoint Interrupt (INT 3)**

  The type 3 interrupt is used to implement break point function in the system. The type 3 interrupt is produced by execution of the INT 3 instruction. Break point function is often used as a debugging aid in cases where single stepping provides more detail than wanted. When we insert a breakpoint, the system executes the instructions up to the breakpoint, and then goes to the breakpoint procedure. In the break point procedure we can write a program to display register contents, memory contents and other information that is required to debug our program. We can insert as many breakpoints as we want in our program.

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

- **Type 4 : Overflow Interrupt (INT O)**

  The type 4 interrupt is used to check overflow condition after any signed arithmetic operation in the system. The 8086 overflow flag (OF), will be set if the signed result of an arithmetic operation on two signed numbers is too large to be represented in the destination register or memory location.

We can prioritize the 8086 interrupts as follows:

Table 5.5: 8086 interrupt priority

Interrupt	Priority
Divide Error, INT(n), INTO	Highest
NMI	↓
INTR	↓
Single Step	Lowest

Interrupt Vector Table (IVT)

.The *interrupt vector table* is a feature of the Intel 8086 family of microprocessors. An *interrupt vector* is a 4-byte number stored in the first 1024 bytes of the memory (00000H – 003FFH). There are 256 different interrupt vectors. Each vector contains the address of an interrupt service routine. Each vector contains a value for IP and CS that forms the address of the interrupt service routine. The first two bytes contain the IP, and the last two bytes contain the CS. Thus, *interrupt vector table* (IVT) is a 1024 bytes sized table that contains the addresses of interrupt service routine. The purpose of the IVT is to hold the vectors that redirect the microprocessor to the right place when an interrupt arrives.

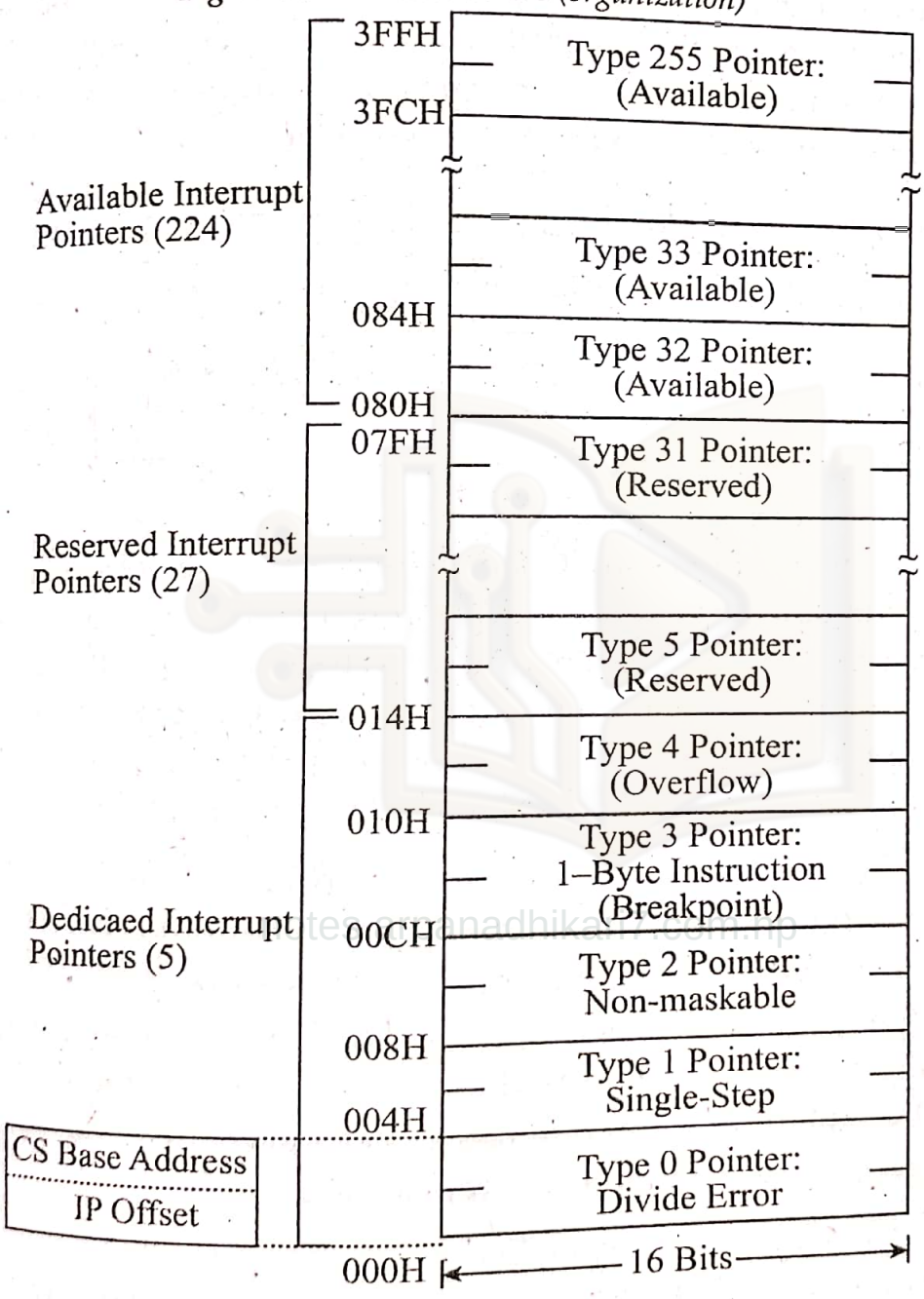
The figure below shows the organization of interrupt vector table. The interrupt number is used as an index into the table to get the address of the interrupt service routine. When the 8086 responds to a particular type interrupt, it automatically multiplies the type by 4 to produce the desired address in the interrupt vector table. It then goes to that address in the table to get the starting address of the interrupt service routine.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

The lowest five types (type 0 to type 4) are dedicated to specific interrupts such as the divide-by-zero interrupt, the single-step interrupt, the non-maskable interrupt, the breakpoint interrupt, and the overflow interrupt. Interrupt types 5 to 31 are reserved by Intel for use in more complex microprocessors. The upper 224 interrupt types from 32 to 255 are available for us to use for hardware or software interrupts.

Figure 5.14: IVT structure (organization)



Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

DOS & BIOS interrupts:

Dos interrupts services link applications with os services such as opening file, reading, writing content using certain functions of INT 4H. BIOS interrupts control the screen disk controller and keyboard operation using INT 10H, 13H,16H etc.

5.9 Priority Interrupt Controller (PIC)

The INTR pin can be used for multiple peripherals, and to determine priorities among these devices when two or more peripherals request interrupt service simultaneously, *priority interrupt controller* (PIC) is used. The PIC includes a status register, a priority comparator, and a priority encoder.

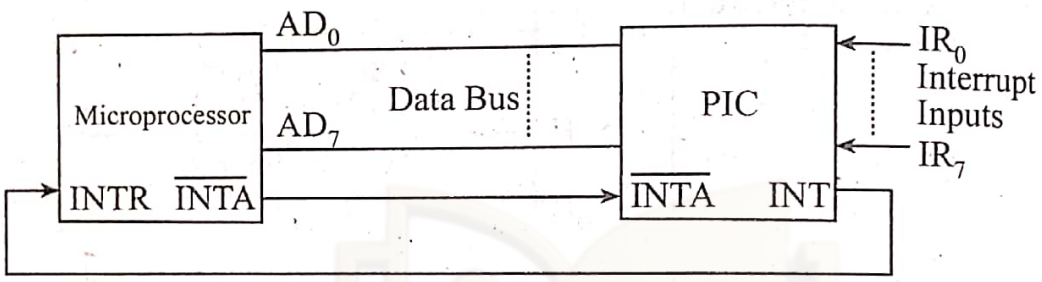


Figure 5.15 Multiple interrupts using PIC

If there are simultaneous requests, the priorities are determined by the priority encoder; it responds to the higher level input, ignoring the lower level input. The drawback of this scheme is that the interrupting device connected to input IR_7 always has the highest priority. This drawback has been solved with the advent of 8259A programmable interrupt controller.

Programmable interrupt controller (PIC) is a device that is used to combine several sources of interrupt onto one or more CPU lines, while allowing priority levels to be assigned to its interrupt outputs. When the device has multiple interrupt outputs to assert, it asserts them in the order of their relative priority. This 8259A programmable interrupt controller is specially designed to work with Intel microprocessor 8085 and 8086.

The 8259A has eight interrupt request inputs IR_7-IR_0. The 8259A uses its INT output to interrupt the microprocessor through INTR pin. The 8259A receives interrupt acknowledge pulses from

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

the microprocessor at its $\overline{INTA}$ input. Vector address, used by the microprocessor to transfer control to the service subroutine of the interrupting device, is provided by the 8259A on the data bus. Below figure shows the internal architecture of 8259A PIC.

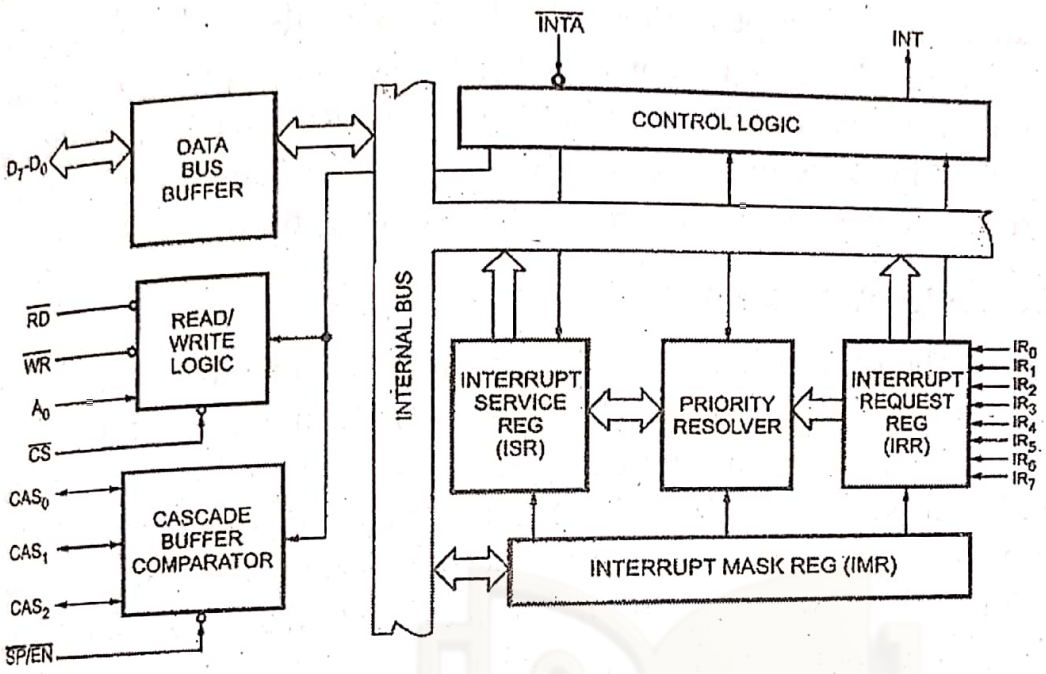


Figure 5.16 Intel 8259A programmable interrupt controller

Data bus buffer

This bidirectional 8-bit buffer is used to interface the 8259A to the microprocessor data bus.

Read/Write & Control Logic

It accepts output commands sent from the microprocessor. It contains the initialization command word (ICW) registers and operation command word (OCW) registers which store the various control formats for device operation. It also allows the status of 8259A to be transferred to the data bus.

Interrupt Request Register (IRR)

It stores all the interrupt inputs that are requesting service. It is an 8-bit register; one bit for each interrupt request. Basically, it keeps track of which interrupt inputs are asking for service. If an interrupt input is unmasked, and has an interrupt signal on it, then

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

the corresponding bit in the IRR will be set. The content of this register can be read to know the status of pending interrupts.

Interrupt Mask Register (IMR)

The IMR is used to disable (Mask) or enable (Unmask) individual interrupt request inputs. This is also an 8-bit register. Each bit in this register corresponds to the interrupt input with the same number. The IMR operates on the IRR. Masking of higher priority input will not affect the interrupt request lines of lower priority. To unmask any interrupt, the corresponding bit is set '0'.

In-service Register (ISR)

The in-service register keeps track of which interrupt inputs are currently being serviced. In 8259A, during the service of an interrupt request, if another higher priority interrupt becomes active, it will be acknowledged and the control will be transferred from lower priority interrupt service subroutine (ISS) to higher priority ISS. Thus, more than one bit of ISR will be set indicating the number of interrupts being serviced.

Priority Resolver

It determines the priorities of the interrupts set in the IRR. It takes the information from IRR, IMR and ISR to determine whether the new interrupt request is having highest priority or not. If the new interrupt request is having the highest priority, it is selected and processed.

Cascade Buffer/Comparator

notes.arpanadhikari7.com.np

It stores and compares the IDs of all 8259A's in the system. The associated three I/O lines (CAS_2-CAS_0) are outputs when 8259A is used as a master and are inputs when 8259A is used as a slave.



Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Chapter - 6

ADVANCED TOPICS

6.1 Multiprocessing Systems

Traditionally, the computer has been viewed as a sequential machine. Most computer programming languages require the programmer to specify algorithms as sequence of instructions. Processor executes programs by executing machine instructions in a sequence and one at a time. This view of the computer has never been entirely true.

As computer technology has evolved and as the cost of computer hardware has dropped, computer designers have sought more and more opportunities for parallelism, usually to enhance performance and availability. Multiprocessing is an example of parallelism which uses multiple CPUs sharing the common resources such as memory, storage device etc.

Characteristics of Multiprocessing System

- Contains two or more similar general purpose processors of comparable capability.
- All processors share access to common memory.
- All processors share access to I/O devices either through the same channels that provide paths to the same devices.
- System is controlled by an integrated operating system that provides interaction between processors and their programs.

notes.arpanadhikari7.com.np

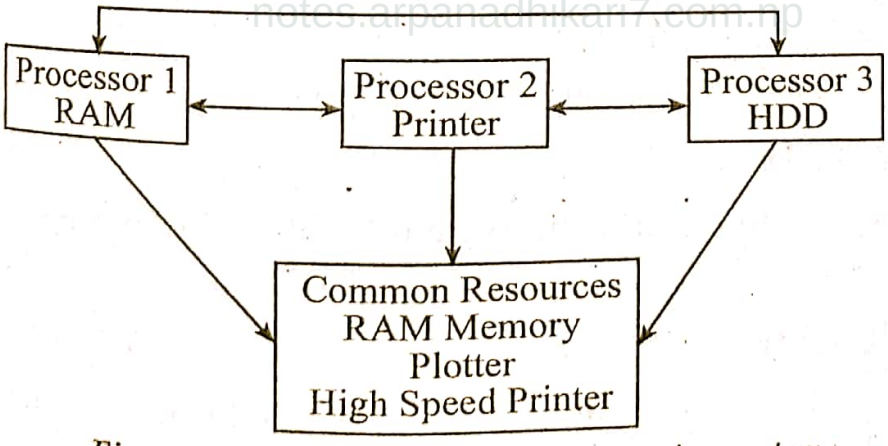


Figure 6.1: Organization of multiprocessing system

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Here, the processors can communicate with each other through memory. The CPUs can directly exchange signals as indicated by dotted line. The organization of multiprocessor system can be divided into three types.

1. Time Shared or Common Bus

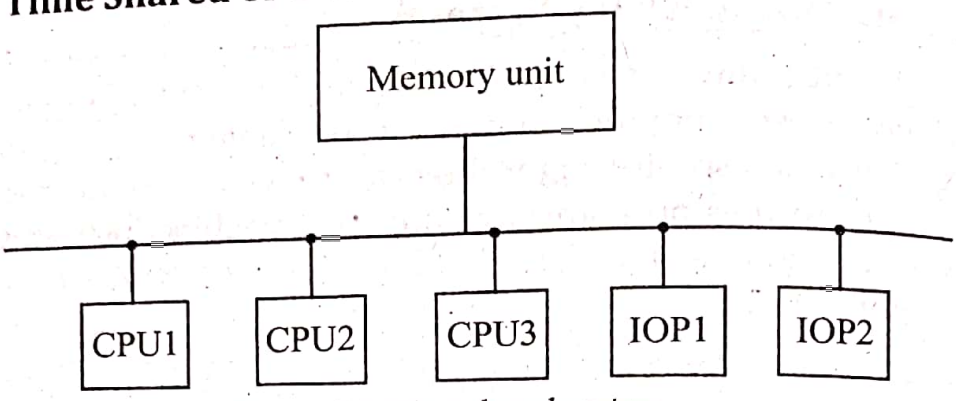


Figure 6.2: Time shared system

There are number of CPUs, I/O modules and memory modules connected to the same bus. So the time shared system must distinguish the modules on the bus to determine the source and destination of the data. Any module in the bus can temporarily act as a master. When one module is controlling the bus, the other should be locked out. The access to each module is divided on the basis of time. The time shared multiprocessing system has the following advantages and drawbacks.

Advantages:

Simplicity

The physical interface and the addressing time sharing logic of each processor remains the same as in a single processor system, so it is very simplest approach.

Flexibility

It is easy to expand the system by attaching more CPUs to the bus.

Reliability

The failure of any attached device should not the failure of the whole system.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Drawbacks:

The speed of the system is limited by the cycle time because all memory references must pass through the common bus.

2. Multiport Memory

Each processor and I/O module has dedicated path to each memory module this system has more performance and complexity than earlier one. For this system, it is possible to configure portions of memory as private to one or more CPUs and/or I/O modules. This feature allows increasing security against unauthorized access, and the storage of recovery routines in areas of memory not susceptible to modification by other processors.

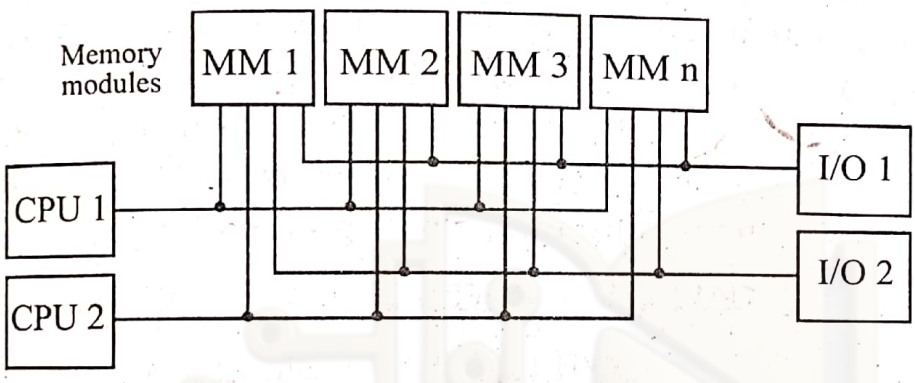


Figure 6.3: Multiport memory system

3. Central Control Unit

It manages the transfer of separate data streams back and forth between independent modules like CPU, memory and I/O. The controller can buffer requests and perform arbitration and timing functions. It can also pass status and control messages between CPUs. All the co-ordination is concentrated in the central control unit un-disturbing the modules. It is more flexible and complex as well.

6.2 Real and Pseudo-Parallelism

In the simplest sense, *parallelism* is the simultaneous use of multiple computer resources to solve a computational problem. In this concept, a problem is broken into discrete parts that can be solved concurrently. Each part is further broken down to a series of instructions. Instructions from each part execute simultaneously on

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

different CPUs. Parallelism can have two different forms: *real-parallelism* and *pseudo-parallelism*.

Real-parallelism consists of the parallel modes of physical devices so that each can carry parallel operations to each other. Two or more processes are actually running at once because the computer system is a parallel processor i.e., has more than one processor.

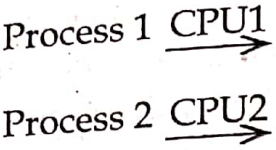


Figure 6.4: True/real parallelism

Pseudo-parallelism consists of the same device carrying the parallel operation. Concurrent processing using parallelism is the pseudo-parallelism which operates either in time division or using other types of parallel algorithms. In pseudo-parallelism, one have the illusion of a parallel processor. Two processes are switched and executed concurrently through a single processor.

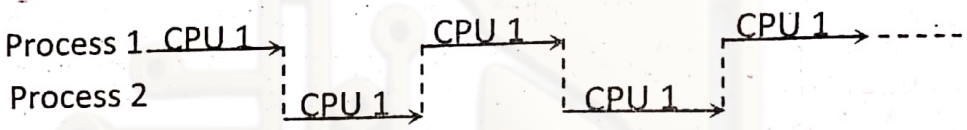


Figure 6.5: Pseudo–parallelism

For example, computer can be reading from a disk and printing on a printer while running a user program. In a multiprogramming system, the CPU switches from program to program, running each program for a fraction of second. Although the CPU is running only one program at any instant of time. As CPU speed is very high so it can work on several programs in a second. It gives user an illusion of parallelism i.e., several processes are being processed at the same time. This rapid switching back and forth of the CPU between programs gives the illusion of parallelism and is termed as *pseudo-parallelism*.

6.3 Flynn's Classification

There are different ways to classify parallel computers. One of the more widely used classifications, in use since 1966, is called

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Flynn's classification. Flynn's classification distinguishes multi-processor computer architectures according to how they can be classified along the two independent dimensions of ***Instruction*** and ***Data***. Each of these dimensions can have only one of two possible states: ***Single*** or ***Multiple***. The matrix below defines the 4 possible classifications according to Flynn:

SISD **Single Instruction, Single Data**	**SIMD** **Single Instruction, Multiple Data**
MISD **Multiple Instruction, Single Data**	**MIMD** **Multiple Instruction, Multiple Data**

1. Single Instruction, Single Data (SISD)

A single processor executes a single instruction stream to operate on data stored in a single memory.



CU: Control Unit
PU: Processing Unit
MU: Memory Unit
IS: Instruction Stream
DS: Data Stream

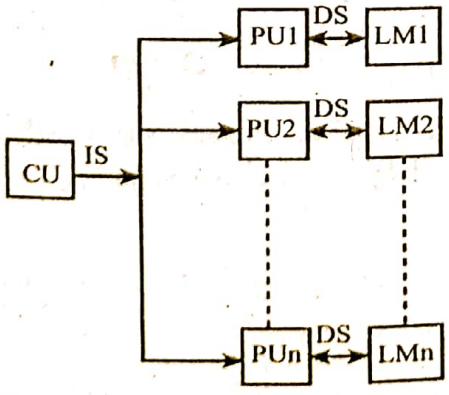
Figure 6.6: SISD

Examples: Uniprocessors such as older generation mainframes, minicomputers and workstations, most modern day PCs.

2. Single Instruction, Multiple Data (SIMD)

Single machine instructions controls the simultaneous execution of a number of processing elements on a lockstep basis. Each processing element has an associated data memory, so that each instruction is executed on a different set of data by the different processors.

notes.arpanadhikari7.com.np



LM: Local Memory

Figure 6.7: SIMD

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Examples:

- Array Processors: Connection Machine CM-2, MasPar MP-1 & MP-2, ILLIAC IV
- Vector Processors : IBM 9000, Cray X-MP, Y-MP & C90, Fujitsu VP, NEC SX-2, Hitachi S820, ETA10

Most modern computers, particularly those with graphics processor units (GPUs) employ SIMD instructions and execution units.

3. Multiple Instruction, Single Data (MISD)

A sequence of data is transmitted to a set of processors, each of which executes a different instruction sequence. MISD structure is only of theoretical interest since no practical systems has been constructed.

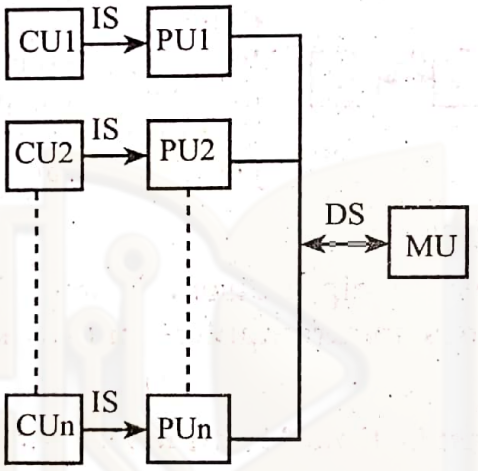


Figure 6.8: MISD

Some conceivable uses of MISD might be:

- Special purpose stream processor (e.g., digital filters)
- Multiple cryptography algorithms attempting to crack a single coded message.

4. Multiple Instruction, Multiple Data (MIMD)

A set of proccessors simultaneously execute different instructions sequences on different data sets.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

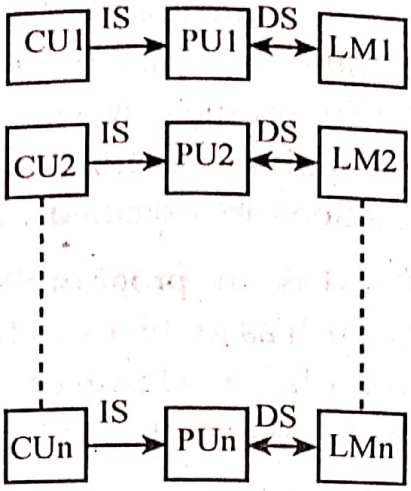


Figure 6.9: MIMD

Examples:

- Most current supercomputers, networked parallel computer clusters and "grids", multi-core PCs which include SMPs (symmetric multiprocessors), NUMA systems.

6.4 Instruction Level, Thread Level and Process Level Parallelism

Instruction-Level Parallelism

Instruction-level parallelism (ILP) is a measure of how many of the operations in a computer program can be performed simultaneously. Consider the following program:

1. e = a + b
2. f = c + d
3. g = e * f

notes.arpanadhikari7.com.np

Operation 3 depends on the results of operations 1 and 2, so it cannot be calculated until both of them are completed. However, operations 1 and 2 do not depend on any other operation, so they can be calculated simultaneously. If we assume that each operation can be completed in one unit of time, then these three instructions can be completed in a total of two units of time, giving an ILP of 3/2.

A goal of compiler and processor designers is to identify and take advantage of as much ILP as possible. Ordinary programs are

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

typically written under a sequential execution model where instructions execute one after the other and in the order specified by the programmer. ILP allows the compiler and the processor to overlap the execution of multiple instructions or even to change the order in which instructions are executed.

How much ILP exists in programs is very application specific. In certain fields, such as graphics and scientific computing the amount can be very large. However, workloads such as cryptography exhibit much less parallelism.

Micro-architectural techniques that are used to exploit ILP include:

- **Instruction pipelining** where the execution of multiple instructions can be partially overlapped.
- **Superscalar execution, VLIW, and the closely related Explicitly Parallel Instruction Computing** concepts, in which multiple execution units are used to execute multiple instructions in parallel.

Thread-Level Parallelism

Thread parallelism (also known as *task parallelism, function parallelism* and *control parallelism*) is a form of parallelization of computer code across multiple processors in parallel environments. Thread parallelism focuses on distributing execution processes (threads) across different parallel computing nodes. It contrasts to data parallelism as another form of parallelism.

It was later recognized that finer-grain parallelism existed with a single program. A single program might have several threads (or functions) that could be executed separately or in parallel. Some of the earliest examples of this technology implemented input/output processing such as direct memory access as a separate thread from the computation thread. A more general approach to this technology was introduced in the 1970s when systems were designed to run multiple computation threads in parallel. This technology is known as multi-threading (MT).

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

As a simple example, if we are running code on a 2-processor system (CPUs "a" & "b") in a parallel environment and we wish to do tasks "A" and "B" , it is possible to tell CPU "a" to do task "A" and CPU "b" to do task 'B" simultaneously, thereby reducing the run time of the execution.

Thread parallelism emphasizes the distributed (parallelized) nature of the processing (i.e., threads), as opposed to the data (data parallelism). Most real programs fall somewhere on a continuum between thread parallelism and data parallelism.

The pseudocode below illustrates task parallelism:

```
program:
...
if CPU="a" then
  do task "A"
else if CPU="b" then
  do task "B"
end if
...
end program
```

The goal of the program is to do some net total task ("A+B"). If we write the code as above and launch it on a 2-processor system, then the runtime environment will execute it as follows.

- In an SPMD system, both CPUs will execute the code.
- In a parallel environment, both will have access to the same data.
- The "if" clause differentiates between the CPU's. CPU "a" will read true on the "if" and CPU "b" will read true on the "else if", thus having their own task.
- Now, both CPU's execute separate code blocks simultaneously, performing different tasks simultaneously.

Code executed by CPU "a":

```
program:
...
```

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

```
    do task "A"
    ...
    end program
Code executed by CPU "b":
    program:
    ...
    do task "B"
    ...
    end program
```

This concept can now be generalized to any number of processors.

Data Parallelism

Data parallelism is parallelism inherent in program loops, which focuses on distributing the data across different computing nodes to be processed in parallel. Parallelizing loops often leads to similar (not necessarily identical) operation sequences or functions being performed on elements of a large data structure. Many scientific and engineering applications exhibit data parallelism.

A loop-carried dependency is the dependence of a loop iteration on the output of one or more previous iterations. Loop-carried dependencies prevent the parallelization of loops. For example, consider the following pseudocode that computes the first few Fibonacci numbers:

```
PREV1 := 0
PREV2 := 1
do:
  CUR := PREV1 + PREV2
  PREV1 := PREV2
  PREV2 := CUR
while (CUR < 10)
```

notes.arpanadhikari7.com.np

This loop cannot be parallelized because CUR depends on itself (PREV2) and PREV1, which are computed in each loop

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

iteration. Since each iteration depends on the result of the previous one, they cannot be performed in parallel. As the size of a problem gets bigger, the amount of data-parallelism available usually does as well.

Process-Level Parallelism

Process-level parallelism is the use of one or more central processing units (CPUs) within a single computer system. The term also refers to the ability of a system to support more than one processor and/or the ability to allocate tasks between them. There are many variations on this basic theme, and the definition of process level parallelism can vary with context, mostly as a function of how CPUs are defined (multiple cores on one die, multiple dies in one package, multiple packages in one system unit, etc.).

This varies, depending upon who you talk to. In the past, a CPU (Central Processing Unit) was a singular execution component for a computer. Then, multiple CPUs were incorporated into a node. Then, individual CPUs were subdivided into multiple "cores", each being a unique execution unit. CPUs with multiple cores are sometimes called "sockets". The result is a node with multiple CPUs, each containing multiple cores.

- During the past 20+ years, the trends indicated by ever faster networks, distributed systems, and multi-processor computer architectures (even at the desktop level) clearly show that parallelism is the future of computing.
- In this same time period, there has been a greater than 1000x increase in supercomputer performance, with no end currently in sight.

6.5 Inter-process Communication, Resource Allocation, and Deadlock

Inter-Process Communication

In computing, *inter-process communication* (IPC) is a set of methods for the exchange of data among multiple threads in one or more processes. Processes may be running on one or more

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

computers connected by a network. IPC methods are divided into methods for message passing, synchronization, shared memory, and remote procedure calls (RPC). The method of IPC used may vary based on the bandwidth and latency of communication between the threads, and the type of data being communicated.

There are several reasons for providing an environment that allows process cooperation:

- Information sharing
- Speedup
- Modularity
- Convenience
- Privilege separation

IPC may also be referred to as inter-thread communication and inter-application communication.

The combination of IPC with the address space concept is the foundation for address space independence/isolation.

The single operating system controls the use of system resources in a multiprocessing environment. In this system, multiple jobs or process may be active at one time. The responsibility of operating system or system software is to schedule the execution and to allocate resources. The functions of multiprocessor operating system are:

- An interface between users and machine
- Resource management
- Memory management
- Prevent deadlocks
- Abnormal program termination
- Process scheduling
- Managers security

notes.arpanadhikari7.com.np

Resource Allocation

In computing, *resource allocation* is necessary for any application to be run on the system. When the user opens any program, this will be counted as a process, and therefore, requires

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

the computer to allocate certain resources for it to be able to run. Such resources could be access to a section of the computer's memory, data in a device interface buffer, one or more files, or the required amount of processing power.

A computer with a single processor can only perform one process at a time, regardless of the amount of programs loaded by the user (or initiated on start-up). Computers using single processors appear to be running multiple programs at once because the processor quickly alternates between programs, processing what is needed in very small amount of time. This process is known as multitasking or *time slicing*. The time allocation is automatic, however higher or lower priority may be given to certain processes, essentially giving high priority programs more/bigger **slices** of the processor's time.

On a computer with multiple processors, different processes can be allocated to different processors so that the computer can truly multitask. Some programs, such as Adobe Photoshop, which can require intense processing power, have been coded so that they are able to run on more than one processor at once, thus running more quickly and efficiently.

Deadlock

A process requests resources; if the resources are not available at that time, the process enters a wait state. Waiting processes may never again change state, because the resources they have requested are held by other waiting processes. This situation is called a *deadlock*.

Processes need access to resources in reasonable order. Suppose a process holds resource A and requests resource B, at same time another process holds B and requests A; both are blocked and remain in deadlock.

A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause. Usually the event is release of a currently held resource. None of the processes can run, release resources and then be awakened.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Deadlock can be studied under two categories:

(a) process deadlock (b) system deadlock.

Conditions for deadlock:

(i) **Mutual exclusion**

Shared resources are used in a mutually exclusive manner.

(ii) **Hold and wait**

Processes hold onto resources they already have while waiting for allocation of other resources.

(iii) **No preemption**

Resources cannot be preempted until the process release them.

(iv) **Circular wait**

Circular chain of processes exist in which each process holds resources wanted by the next process in chain.

6.6 Operating System

An *operating system* is a collection of software that provides services for computer programs. In other terms, an operating system is a composition of a kernel and utility programs; the kernel controls the allocation of hardware resources while the utility programs enhance the usefulness of the computer. An operating system kernel is a computer program that serves as an intermediary layer between the hardware and application programs. A kernel makes it possible for software to interact with the underlying hardware of the operating system. Such software (mostly application software) achieves this by issuing service requests to the kernel. (These requests are called system calls.) When a service request is received, the kernel translates it into instructions for the CPU or other electronic components of the computer to execute.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

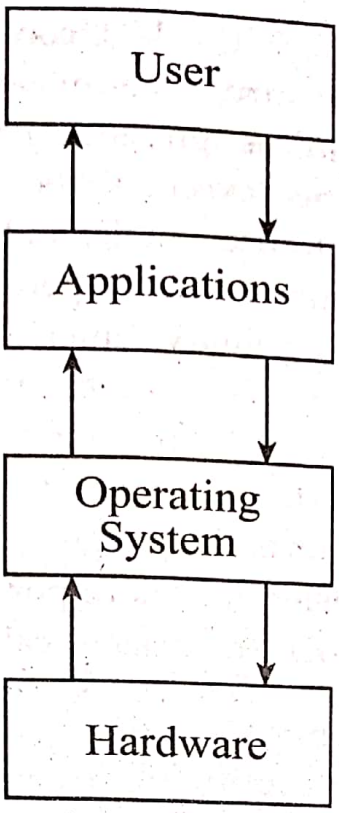


Figure 6.10: Operating system

Features of operating system:

When talking about features of an operating system, often they get mixed up with its functions. We define an operating system's feature as a prominent attribute of the operating system; in other words, its major components.

i. **System calls**

On modern microprocessors, there are at least two modes of operation: kernel mode and user mode. If an application running in user mode tries to perform a privileged operation (such as directly accessing the hardware), the CPU will most likely throw an exception. So, then, how does an application read input from the keyboard or write to the screen? It does so by sending a request to the kernel. Of course, this slows down the operation, but ensures that application programs do not execute code that could damage or compromise the system. These requests that application programs send to the kernel are called *system calls*. A **system call** can thus be simply defined as a request by a computer program to the operating system's kernel.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

When a system call is invoked, control is transferred to the kernel which, in turn, determines whether the calling application should be granted the requested service. If granted, the kernel executes the necessary instructions, causes a switch into user mode, and returns control back to the calling program. Most operating systems however provide an intermediary interface that sits between applications and the system calls layer, in a form of a library or an Application Programming Interface (API). Such an intermediary interface makes it possible for programs written in high level languages to invoke system calls. Moreover, it is easier and more portable to use a library or an API than to code the system call in assembly language instructions.

ii. Device drivers

A computer system is usually made up of several devices such as disk drives, keyboards, mice, video adapters, sound cards, etc. When a user attaches such devices to their computer, they expect the operating system to identify the device and make use of it. Indeed, the operating system may know *what* the device is but not *how* to communicate with it. The latter problem is solved by means of a driver. A **device driver** is a computer program that controls a particular device attached to a computer. It provides an interface through which the operating system can transparently make calls to the device. In fact, device drivers have built-in functions that are meant to be called by the operating system or other privileged programs.

notes.arpanadhikari7.com.np

Devices are generally slower compared with the CPU. This means that whereas the CPU could be doing other stuff, it many times waits for a busy but slow device to finish whatever job it is doing. This bad behavior, however, is mitigated by the use of *hardware interrupts*. Interrupts cause control to be transferred to a routine designed to process the interrupt. For example, when a key is pressed on a computer keyboard, a hardware interrupt is generated, which invokes the keyboard device driver. After the driver has finished

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

processing the event, control is returned back to the interrupted program. However, only a few peripherals support interrupts which means that drivers have to poll the hardware, i.e. ask whether there is an event to process.

iii. **File system**

Every computer file is stored in a linear space on a storage device of finite capacity. Each file has its *address* on storage, which is determined by the number of byte offsets from the beginning of the storage medium. But then, there is the need for a structure that tells where one piece of data begins and a where it ends—a file system. File systems keep track of unused space on the disk as well as additional information about each file such as the name, size, owner, creation date, access control, encryption, etc. What is more, file systems manage the directory structure and the mapping of file names to file control blocks. A **file system** can thus be defined as a structured data representation and a set of metadata that describe stored data.

iv. **User interface**

This is another feature of an operating system: its user interface. An operating system's user interface determines how the user interacts with the computer. The two most common forms of a user interface are the Command Line Interface (CLI) and the Graphical User Interface (GUI). A CLI provide a prompt at which commands can be given line-by-line. This kind of interface is usually implemented with a program called a *command line shell*, which accepts commands as text input and converts them to the appropriate operating system functions. CLIs can be quite powerful for experienced users, but if one does not know the system well enough, they can become quite lost. Examples of CLIs are the UNIX shells and the Windows Command Prompt.

In contrast, a GUI provides a visual environment where a device (such as a mouse) is used to navigate the system and perform tasks. Unlike CLIs where performing a task can

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

from accumulator and the out pot of the ALU can be stared in accumulator through data bus. The 8085 microprocessor is an example of accumulator-based architecture.

Register-Based Architecture

Unlike an accumulator-based microprocessor architecture, a register-based microprocessor architecture has one or more general purpose registers. Data can enter into the ALU from any registers. The 8086 microprocessor is an example of register-based architecture.

The advantages of register-based architecture is extendibility and flexibility in programming. However, the disadvantage is requirement of complex circuitry.

6.8 RISC and CISC Architectures

CISC Architecture

Complex instruction set computers is the acronym for CISC and machine based on this architecture have complex instructions. Most of the personal computers which are used today are based on CISC architecture. Following are the characteristics of CISC machines.

- CISC machines have complex instructions which need a large cycles for execution.
- The transfer of data among the register is much faster than among memory and processor. In CISC, various instructions based on memory and processer so the processing speed gets reduced.
- Pipelining is the process of fetching one instruction when another instruction is executing in parallel. Due to complex instruction this feature cannot be heavily used in CISC machines.
- Micro-operations form the instruction and instruction form he micro-program which is written in control memory to perform timing and sequencing of the micro-operations implemented in CISC.

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

- CISC machines have large number of complex instructions based on multiple numbers of addressing modes.
- CISC machines processer does not consist of large number of registers due to large cost. So these machines have to perform various memory read and write operations.
- CISC machines are preferable where the speed of processer is not the prime issue and where general applications are to be handled. Processers like 8085, 8086, 8086,8086,8086,8086 etc are based on CISC processers and even today's pc.

RISC Architecture

The term RISC represents reduced instruction set computing/computers. It focuses on a small set of instructions which simplifies the hardware design and improves the processer performance. Generally, RISC processers include the following features.

- The number of instructions is minimized i.e., less than 100 and each can be executed in a single clock cycle the data path cycle time is the time required to fetch the operations from the registers, run them through ALU and store the result back to register which is very small in RISC.
- the number of addressing modes is relatively low i.e. less than 3 and only few instructions are memory referencing link lode and store so that RISC permits heavy pipelining.
- The processing is register intensive, meaning it includes many more registers and most of the computing is performed using registers range from 32 registers to more than 100 registers.
- There is no micro-programs in RISC machines for interpreting the instructions. The most instructions are directly executed by the hardware.
- Design considerations include support for high level languages through appropriate selection of instruction and optimization of compilers.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

- Usually in scientific and research oriented tasks where the speed of processers is apex issue, RISC machines are used and replacing CISC now due to reduce the price of hardware.
- RISC based system are R4400SC from MIPS, PA7100 from HP, Power PC from Apple, IBM and Motorola, super sparc from sun, i860™ from Intel etc.

Differences between RISC and CISC architecture:

RISC	CISC
1. Simple instructions taking one cycle.	1. Complex instructions taking multiple no. of cycles.
2. Most operations are register to register with only LOAD and STORE operations accessing memory.	2. In CISC machine, memory read & write operations form inherent part for executing the instructions.
3. Heavily pipelined, increasing performance.	3. Not/less pipelined
4. Multiple register sets.	4. Single register set
5. Instructions have fixed format.	5. Instructions have variable format.
6. Few number of instructions and addressing modes.	6. Large number of instructions and addressing modes.
7. Complexity is in the design of compiler.	7. Complexity is in microprogram.
8. Instructions are executed by hardware.	8. Instructions in CISC machine are interpreted by microprogram.
9. Since RISC processors are simpler than CISC processors, they can be designed more quickly.	9. CISC processors have comparatively longer design cycle.
10. Examples include SPARC, MIPS R4000, etc.	10. Examples include IBM 370/168, VAX 11/780, 8085, 8086, 80286, 80386, etc.

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

6.9 The Digital Signal Processor (DSP)

A *digital signal processor* (DSP) is a specialized microprocessor with an architecture optimized for the fast operational needs of digital signal processing. Essentially, a DSP is a special type of microprocessor that processes data in real time. Its applications focus on the processing of digital data that represents analog signals. A DSP, like a microprocessor, has a central processing unit (CPU) and memory units in addition to many interfacing functions. Every time, we use our cellular telephone, we are using a DSP, and this is only one example of its many applications.

The digital signal processor (DSP) is the heart of a digital signal processing system. It takes its input from an ADC and produces an output that goes to a DAC, as shown in Figure 6.11.

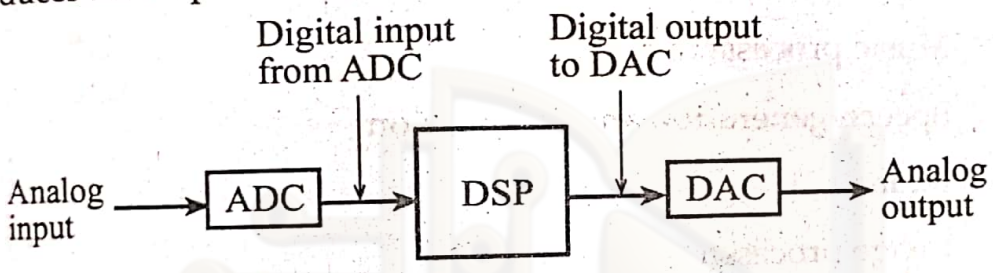


Figure 6.11: The DSP has a digital input and produces a digital output.

DSPs are typically programmed in either assembly language or in C. Because programs written in assembly language can usually execute faster and because speed is critical in most DSP applications, assembly language is used much more in DSPs than in general-purpose microprocessors. Also, DSP programs are usually much shorter than traditional microprocessor programs because of their very specialized applications where much redundancy is used. In general, the instruction sets for DSPs tend to be smaller than for microprocessors.

DSP applications:

The DSP, unlike the general-purpose microprocessor, must typically process data in real time; that is, as it happens. Many applications in which DSPs are used cannot tolerate any noticeable delays, requiring the DSP to be extremely fast. In addition to cell

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

phones, digital signal processors (DSPs) are used in multimedia computers, video recorders, CD players, hard disk drives, digital radio modems, and other applications to improve the signal quality. DSPs are also used in television applications. For example, television converters use DSP to provide compatibility with various television standards.

An important application of DSPs is in signal compression and decompression. In CD systems, for example, the music on the CD is in a compressed form so that it doesn't use as much storage space. It must be decompressed in order to be reproduced. Also signal compression is used in cell phones to allow a greater number of calls to be handled simultaneously in a local cell.

Application areas of DSPs are:

1. Telecommunications
2. Music processing
3. Speech generation and recognition
4. Radar
5. Image processing
6. Implementation of digital filters



notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

APPENDIX

List of 8085 Instructions

Instruction	Bytes	Machine Cycles	T-States
ACI 8-bit data	2	2	7
ADC Reg.	1	1	4
ADC Mem.	1	2	7
ADD Reg.	1	1	4
ADD Mem.	1	2	7
ADI 8-bit data	2	2	7
ANA Reg.	1	1	4
ANA Mem.	1	2	7
ANI 8-bit data	2	2	7
CALL 16-bit address (unconditional)	3	5	18
CMA	1	1	4
CMC	1	1	4
CMP Reg.	1	1	4
CMP Mem.	1	2	7
CPI 8-bit data	2	2	7
DAA	1	1	4
DAD Reg. pair	1	3	10
DCR Reg.	1	1	4
DCR Mem.	1	3	10
DCX Reg. pair	1	1	6
DI	1	1	4
EI	1	1	4

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Instruction	Bytes	Machine Cycles	T-States
HLT	1	2 or more	5 or more
IN 8-bit port address	2	3	10
INR Reg.	1	1	4
INR Mem.	1	3	10
INX Reg. pair	1	1	6
JMP 16-bit address (unconditional)	3	3	10
LDA 16-bit address	3	4	13
LDAX B/D reg. pair	1	2	7
LHLD 16-bit address	3	5	16
LXI Reg. pair,16-bit data	3	3	10
MOV Reg.,Reg.	1	1	4
MOV Mem.,Reg.	1	2	7
MOV Reg.,Mem.	1	2	7
MVI Reg.,Data	2	2	7
MVI Mem.,Data	2	3	10
NOP	1	1	4
ORA Reg.	1	1	4
ORA Mem.	1	2	7
ORI 8-bit data	2	2	7
OUT 8-bit port address	2	3	10
PCHL	1	1	6
POP Reg. pair	1	3	10
PUSH Reg. pair	1	3	12
RAL	1	1	4

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np

Instruction	Bytes	Machine Cycles	T-States
RAR	1	1	4
RLC	1	1	4
RRC	1	1	4
RET (unconditional)	1	3	10
RIM	1	1	4
RST	1	3	12
SBB Reg.	1	1	4
SBB Mem.	1	2	7
SBI 8-bit data	2	2	7
SHLD 16-bit address	3	5	16
SIM	1	1	4
SPHL	1	1	6
STA 16-bit address	3	4	13
STAX B/D reg. pair	1	2	7
STC	1	1	4
SUB Reg.	1	1	4
SUB Mem.	1	2	7
SUI 8-bit data	2	2	7
XCHG	1	1	4
XRA Reg.	1	1	4
XRA Mem.	1	2	7
XRI 8-bit data	2	2	7
XTHL	1	5	16

notes.arpanadhikari7.com.np

Scanned with CamScanner

Available on notes.arpanadhikari7.com.np